



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks

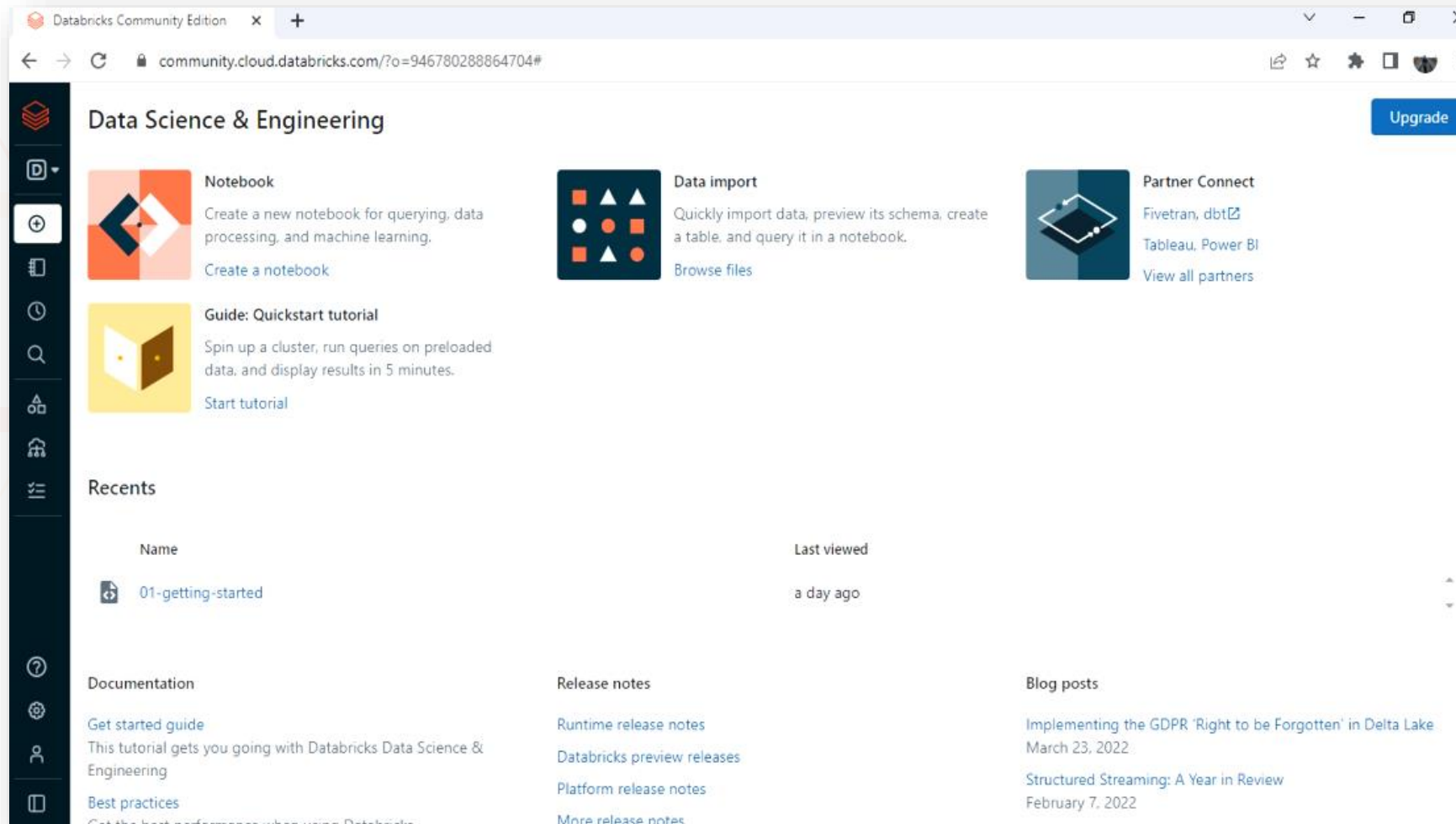


Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Creating Spark Tables

Start your Databricks Workspace.

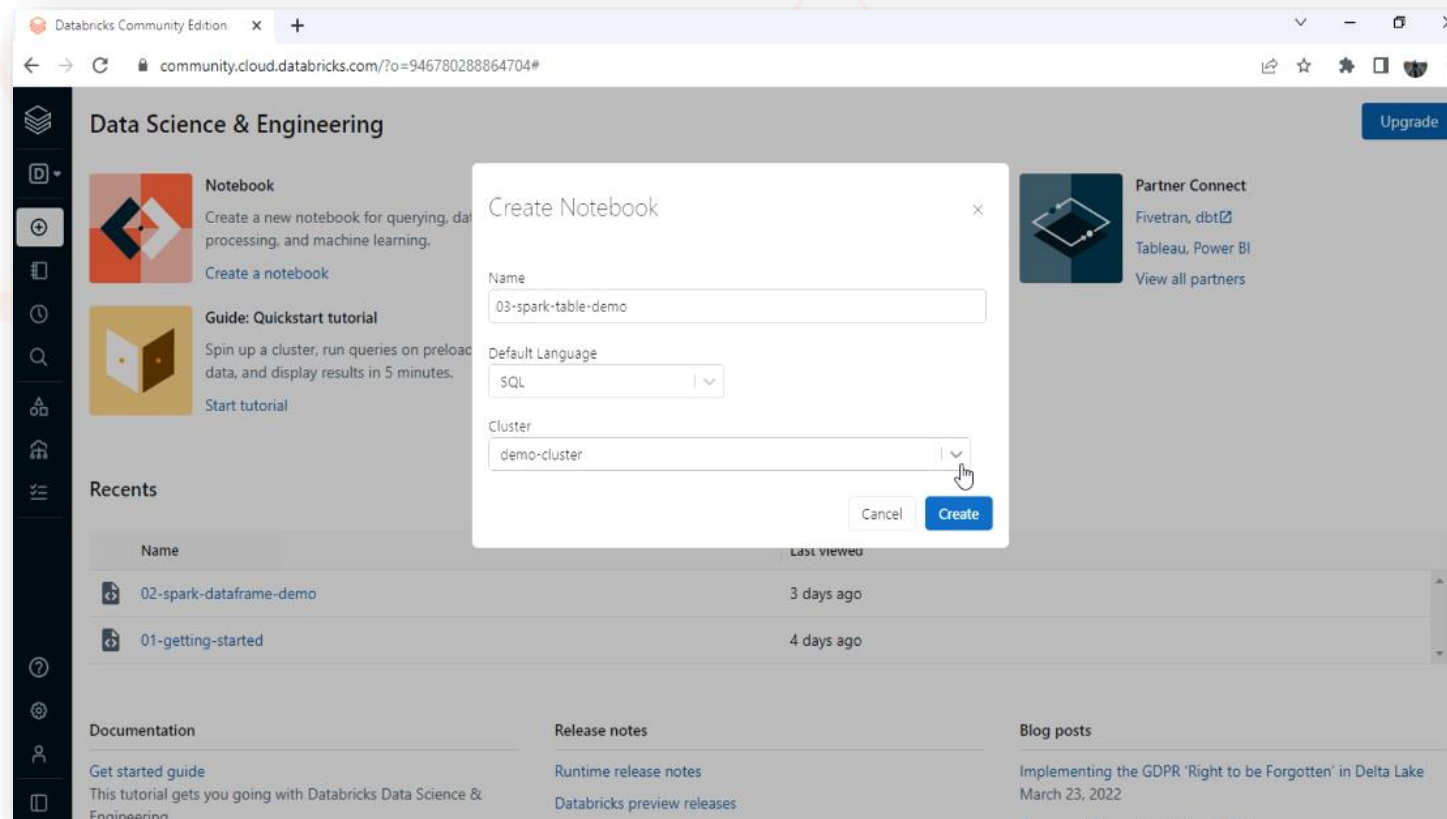


Create a new notebook. (**Reference: 03-spark-table-demo.sql**)

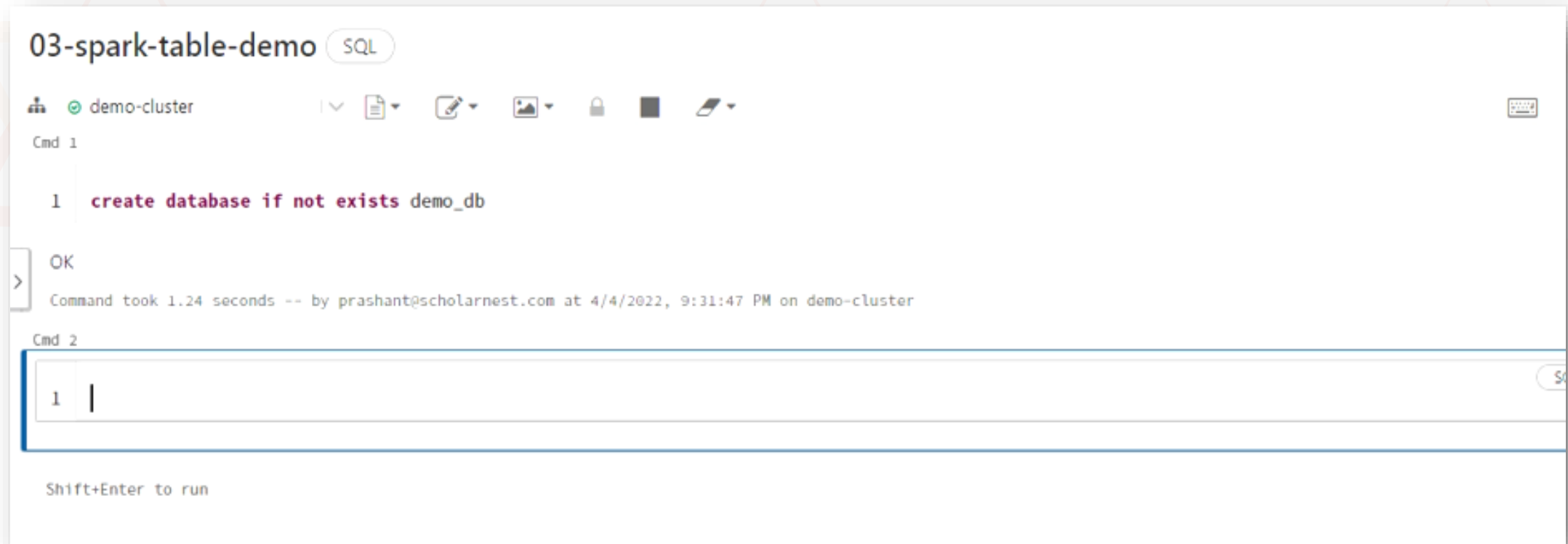
Give a name to your notebook.

Choose your notebook's default language. This time, we want to use Spark SQL, so the default language for the notebook should be SQL.

Attach a cluster to your notebook. If you do not have a running cluster, go back to the Compute menu and create a new cluster.



Create a database as shown below.
The code shown here is purely a SQL DDL Statement. A database should have been created.



The screenshot displays the Databricks SQL interface for a workspace named '03-spark-table-demo'. The interface includes a toolbar with icons for cluster management, document management, and other SQL-specific actions. The main area shows the execution of a SQL command. The command is: `1 create database if not exists demo_db`. The execution status is 'OK', and a message indicates the command took 1.24 seconds and was executed by 'prashant@scholarnest.com' on '4/4/2022, 9:31:47 PM' on the 'demo-cluster'. Below the command area, there is a text input field for a new command, labeled 'Cmd 2', which currently contains the number '1' followed by a cursor. At the bottom, a prompt 'Shift+Enter to run' is visible.

```
03-spark-table-demo SQL
```

demo-cluster

Cmd 1

```
1 create database if not exists demo_db
```

OK

Command took 1.24 seconds -- by prashant@scholarnest.com at 4/4/2022, 9:31:47 PM on demo-cluster

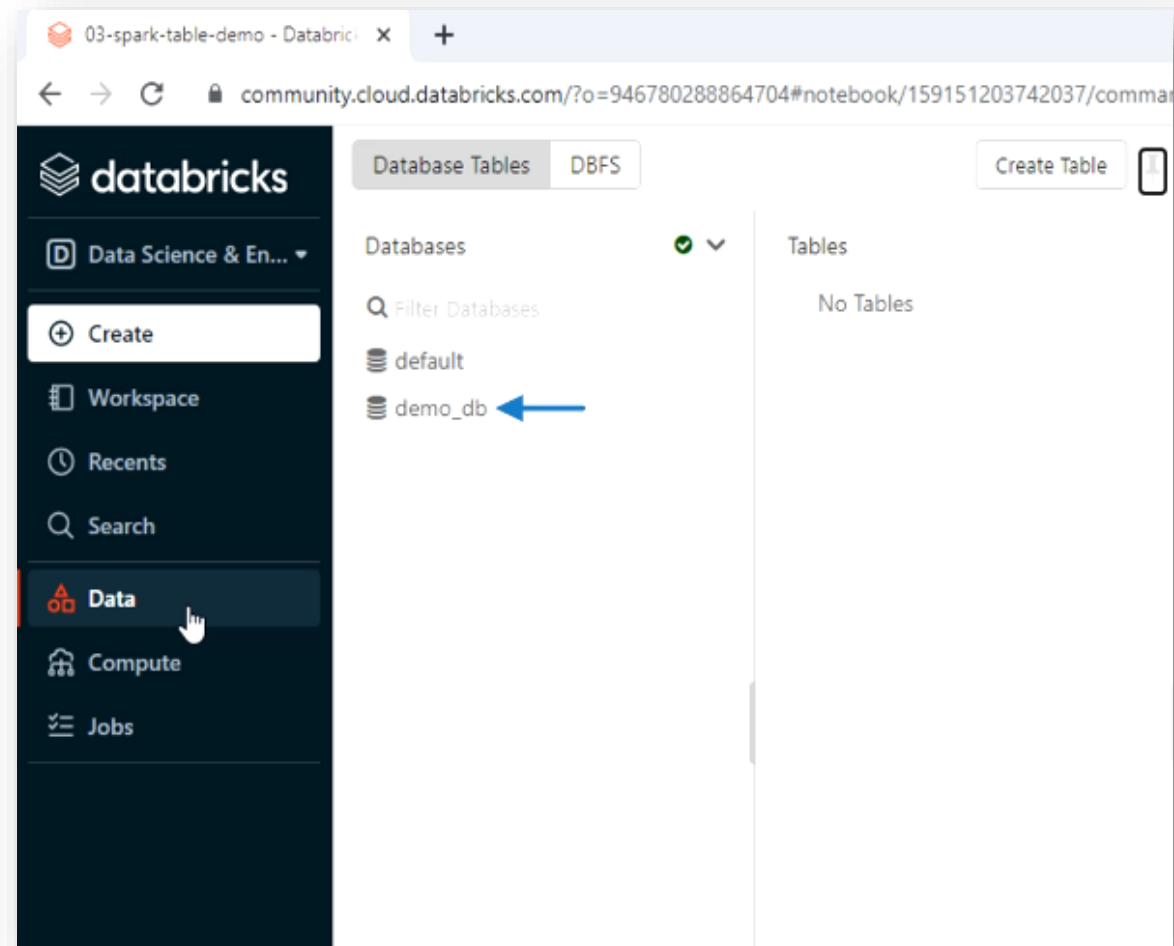
Cmd 2

```
1 |
```

Shift+Enter to run

To see your database, click the Data menu item, and you will see two databases:

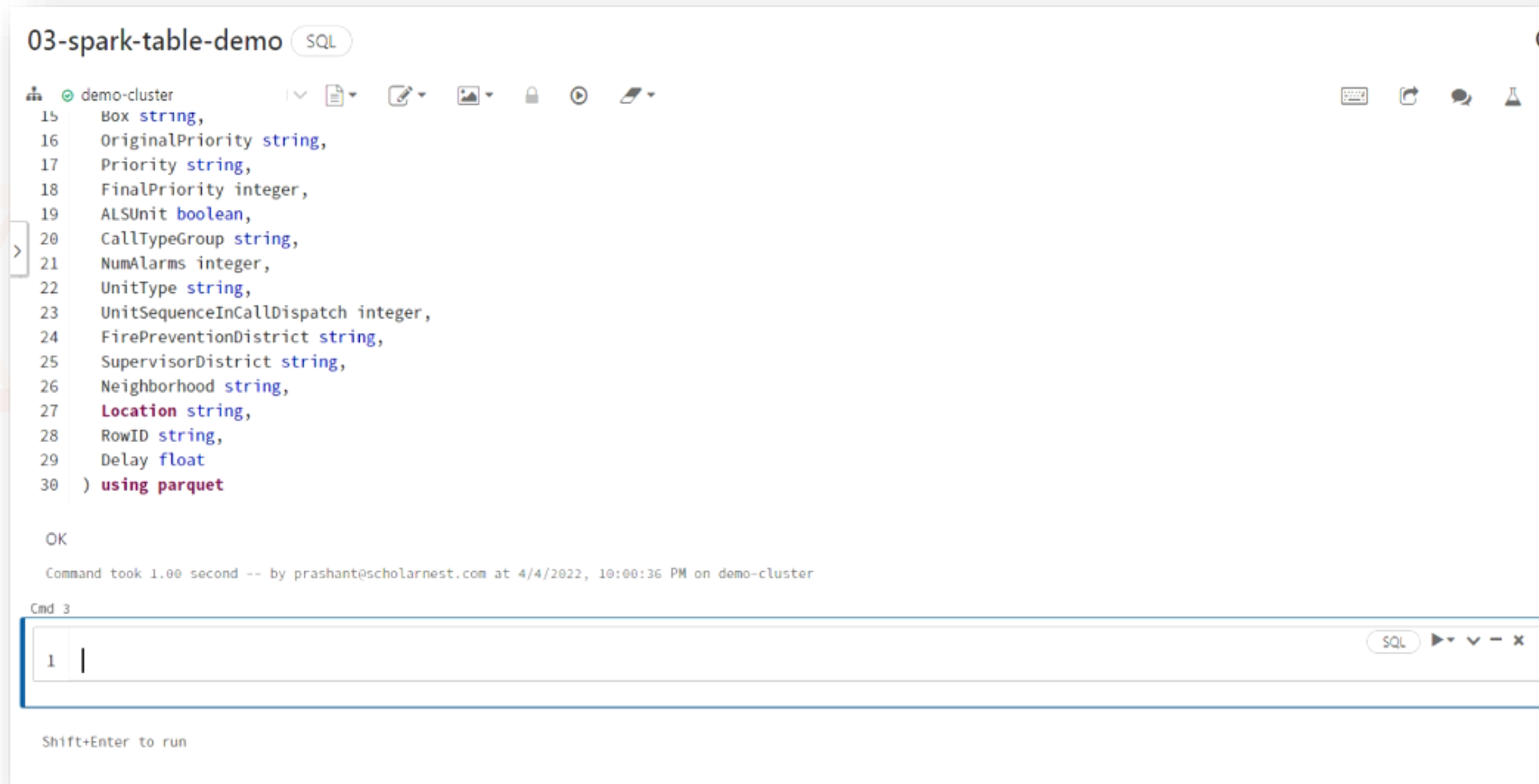
1. Spark created one default database for you. So if you create a table, it will be created inside the default database. But we do not recommend using the default database.
2. Instead, you should create a database for your project and keep all your project tables inside your database. So we have already created one database for ourselves.



Now the next aim is to create a table. The creation of table also follows a create table DDL statement as shown below. We have 28 columns in the fire response call dataset. Most of them are string columns, but some are integer and float columns.

```
03-spark-table-demo SQL
demo-cluster
1 create table if not exists demo_db.fire_service_calls_tbl(
2   CallNumber integer,
3   UnitID string,
4   IncidentNumber integer,
5   CallType string,
6   CallDate string,
7   WatchDate string,
8   CallFinalDisposition string,
9   AvailableDtTm string,
10  Address string,
11  City string,
12  Zipcode integer,
13  Battalion string,
14  StationArea string,
15  Box string,
16  OriginalPriority string,
17  Priority string,
18  FinalPriority integer,
19  ALSUnit boolean,
20  CallTypeGroup string,
21  NumAlarms integer,
22  UnitType string,
23  UnitSequenceInCallDispatch integer,
24  FirePreventionDistrict string,
25  SupervisorDistrict string,
26  Neighborhood string,
27  Location string,
28  RowID string,
29  Delay float
```

Finally, we will specify the file format for the table. I will be using parquet for the table. Every database table internally stores data in files. So for this table, I want Spark to use parquet file format. And if we run this following code, a table would be created for us.



The screenshot shows a Databricks SQL interface with a query editor. The query is a Spark SQL command to create a table named 'Box' with various columns and a parquet file format. The command is as follows:

```
15 Box string,  
16 OriginalPriority string,  
17 Priority string,  
18 FinalPriority integer,  
19 ALSUnit boolean,  
20 CallTypeGroup string,  
21 NumAlarms integer,  
22 UnitType string,  
23 UnitSequenceInCallDispatch integer,  
24 FirePreventionDistrict string,  
25 SupervisorDistrict string,  
26 Neighborhood string,  
27 Location string,  
28 RowID string,  
29 Delay float  
30 ) using parquet
```

Below the query, there is a status bar indicating the command was executed successfully: "Command took 1.00 second -- by prashant@scholarnest.com at 4/4/2022, 10:00:36 PM on demo-cluster". At the bottom, there is a command prompt labeled "Cmd 3" with a cursor and a "Shift+Enter to run" instruction.

We do not have any data in the table. So we can use the insert-into command to insert some records into the table in a traditional database table, as shown in the image below. I am inserting one row into the *demo_db.fire_service_calls_tbl*. All the column values are null except the first column.

03-spark-table-demo

SQL

demo-cluster

28 RowID string,

29 Delay float

30) using parquet

OK

Command took 1.00 second -- by prashant@scholarnest.com at 4/4/2022, 10:00:36 PM on demo-cluster

Cmd 3

1 insert into demo_db.fire_service_calls_tbl ←

2 values(1234, null, null, null, null, null, null, null, null, null, null, null, null, null, null,

3 null, null, null, null, null, null, null, null, null)

▶ (1) Spark Jobs

OK

Command took 8.37 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:01:52 PM on demo-cluster

Cmd 4

You can run a select * expression to see the content of the table. And we can see in the output that data is inserted successfully.

03-spark-table-demo SQL

demo-cluster

Cmd 4

```
1 select * from demo_db.fire_service_calls_tbl
```

▶ (1) Spark Jobs

	CallNumber	UnitID	IncidentNumber	CallType	CallDate	WatchDate	CallFinalDisposition	AvailableDtTm	Address	City	Zipc
1	1234	null	null	null	null	null	null	null	null	null	null

Showing all 1 rows.

Command took 5.79 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:02:34 PM on demo-cluster

Cmd 5

Spark tables are expected to hold millions and billions of records. And we can't write the insert-into values command to load those records.

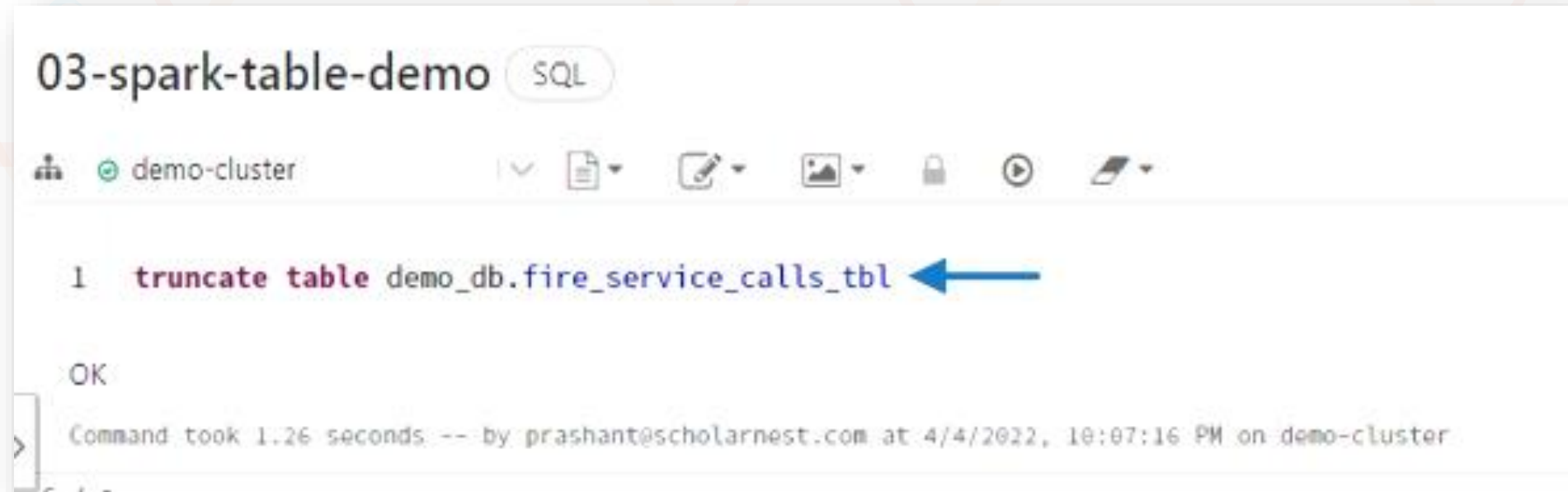
The insert into values expression doesn't make much sense in the big data world.

You will not find a project that uses insert into values to load data into Spark tables.

There are many ways to load data into Spark tables. But for now, we will insert records into the table from another table.

Let us truncate the table, so that we clean the garbage record that we just inserted.

We used the truncate statement because Spark SQL doesn't offer to delete statements.

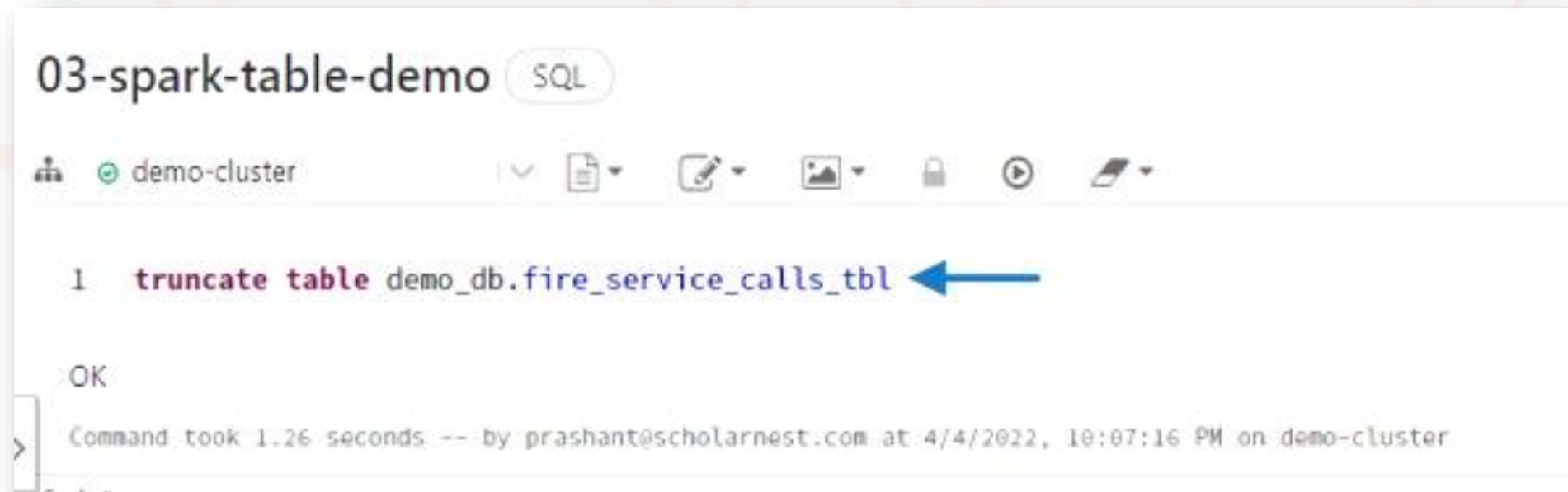


The screenshot shows a Databricks SQL interface. At the top, the title is "03-spark-table-demo" with a "SQL" tab selected. Below the title, there's a toolbar with icons for sharing, saving, and other actions. The main area displays a SQL command: `1 truncate table demo_db.fire_service_calls_tbl`. A blue arrow points to the end of this command. Below the command, there's a status bar that says "OK" and "Command took 1.26 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:07:16 PM on demo-cluster".

```
03-spark-table-demo SQL
demo-cluster
1 truncate table demo_db.fire_service_calls_tbl
OK
Command took 1.26 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:07:16 PM on demo-cluster
```

Let us truncate the table, so that we clean the garbage record that we just inserted.

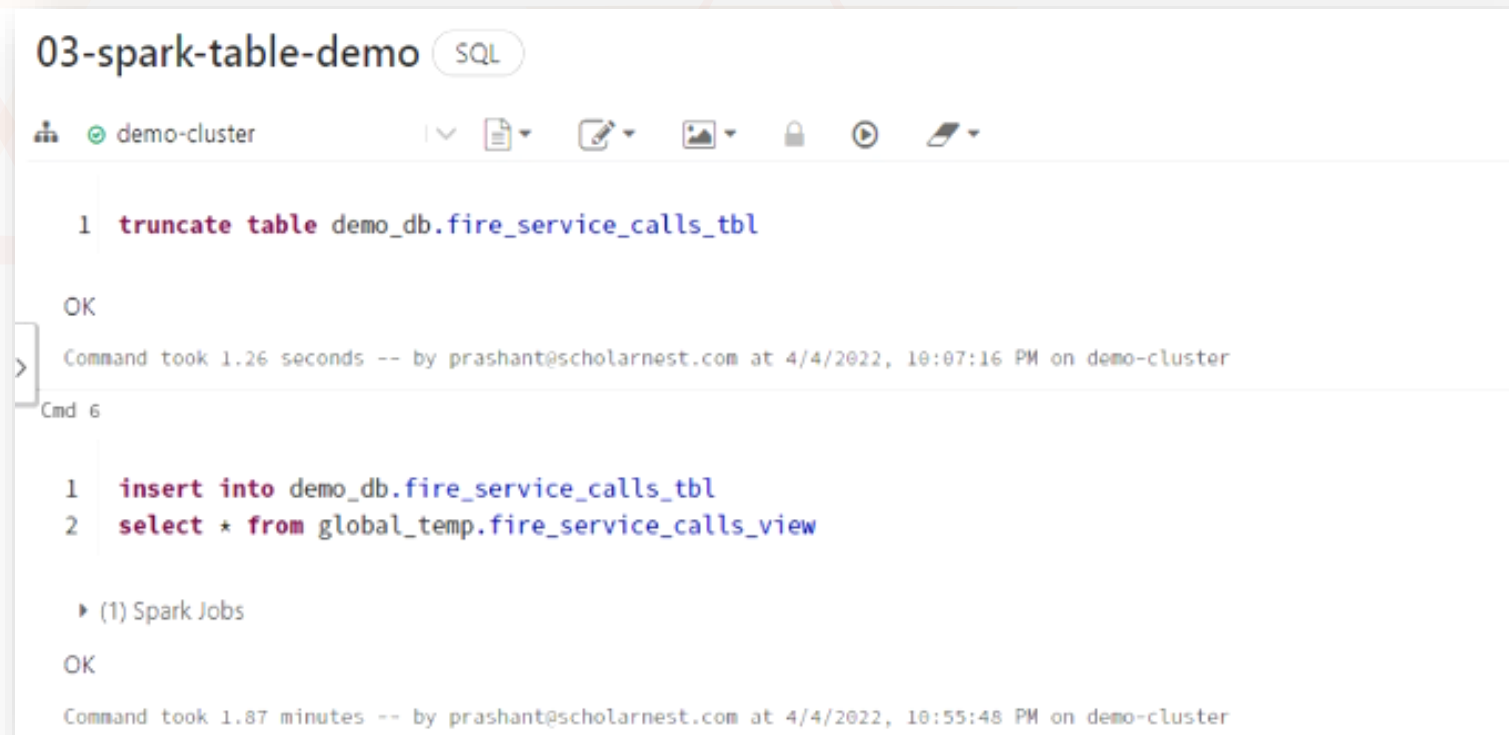
We used the truncate statement because Spark SQL doesn't offer to delete statements. You cannot delete data from a Spark Table using Spark SQL. Databricks does offer a delete expression on Spark tables.



The screenshot shows a Databricks SQL interface. At the top, the title is "03-spark-table-demo" with a "SQL" tab selected. Below the title, there is a toolbar with icons for sharing, cluster selection (showing "demo-cluster"), and various actions like refresh, save, edit, insert, lock, and play. The main area displays a single SQL command: `1 truncate table demo_db.fire_service_calls_tbl`. A blue arrow points to the end of this command. Below the command, there is an "OK" button. At the bottom, a status bar indicates: "Command took 1.26 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:07:16 PM on demo-cluster".

```
03-spark-table-demo SQL
demo-cluster
1 truncate table demo_db.fire_service_calls_tbl
OK
Command took 1.26 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:07:16 PM on demo-cluster
```

The next target is to insert data into the table after it is truncated. And we have the code for the same in the screenshot shown below. I am inserting records into the *demo_db.fire_service_calls_tbl* from *global_temp.fire_service_calls_view*.



03-spark-table-demo SQL

demo-cluster

```
1 truncate table demo_db.fire_service_calls_tbl
```

OK

Command took 1.26 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:07:16 PM on demo-cluster

Cmd 6

```
1 insert into demo_db.fire_service_calls_tbl
2 select * from global_temp.fire_service_calls_view
```

▶ (1) Spark Jobs

OK

Command took 1.87 minutes -- by prashant@scholarnest.com at 4/4/2022, 10:55:48 PM on demo-cluster

Finally, you can see the content of the table using the select expression.

03-spark-table-demo

SQL

demo-cluster

Command took 1.87 minutes -- by prashant@scholarnest.com at 4/4/2022, 10:55:48 PM on demo-cluster

Cmd 7

1

```
select * from demo_db.fire_service_calls_tbl
```

>

(1) Spark Jobs

	CallNumber	UnitID	IncidentNumber	CallType	CallDate	WatchDate	CallFinalDisposition	AvailableDtTm	Address
1	61160285	E22	6032206	Medical Incident	04/26/2006	04/26/2006	Other	null	1400 B
2	61160287	93	6032207	Medical Incident	04/26/2006	04/26/2006	Code 2 Transport	04/26/2006 02:32:37 PM	400 Blk
3	61160287	E13	6032207	Medical Incident	04/26/2006	04/26/2006	Other	04/26/2006 01:40:18 PM	400 Blk
4	61160287	RC4	6032207	Medical Incident	04/26/2006	04/26/2006	Other	04/26/2006 01:47:22 PM	400 Blk
5	61160293	B02	6032209	Alarms	04/26/2006	04/26/2006	Other	04/26/2006 01:31:30 PM	0 Block
6	61160293	E06	6032209	Alarms	04/26/2006	04/26/2006	Other	null	0 Block
7	61160293	T06	6032209	Alarms	04/26/2006	04/26/2006	Other	04/26/2006 01:31:13 PM	0 Block

Truncated results, showing first 1000 rows.

[Click to re-execute with maximum result limits.](#)

Command took 1.89 seconds -- by prashant@scholarnest.com at 4/4/2022, 10:58:31 PM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com